PROGRAMMATION ORIENTEE OBJECT POO

1. Qu'est-ce que la POO ?

- **Définition :**

La POO (Programmation Orientée Objet) est une manière de programmer basée sur des objets.

- **Explication :**

Un programme est construit à partir :

- ☐ de classes (modèles)
- ☐ et d'objets (instances de ces classes)

2. Classe et Objet :

- **Définition :**

Une classe est un modèle qui définit les attributs et les méthodes .

Un objet est une instance d'une classe

- Syntaxe :

```
class NomClasse {
// attributs
// méthodes
}
NomClasse obj = new NomClasse();
```

- Exemple :

```
class Voiture {
        String marque;
        int vitesse;
}
 public class Main {
   public static void main(String[] args) {
            Voiture v1 = new Voiture();
            v1.marque = "Toyota";
            v1.vitesse = 120;
            System.out.println(v1.marque);
    }
}
```

3. Attributs et Méthodes:

• Définition :

- Attribut → variables de la classe
- Méthode → fonctions de la classe

• Exemple :

```
class Personne {
    String nom;
    void direBonjour() {
        System.out.println("Bonjour");
    }
}
```

4. Constructeur:

• Définition :

Un constructeur est une méthode spéciale appelée lors de la création d'un objet.

• Syntax :

```
class Classe {
   Classe() {
       // initialisation
   }
}
```

- Exemple :

```
class Personne {
    String nom;
    Personne(String n) {
        nom = n;
    }
}
```

5. Encapsulation:

- Définition :

L'encapsulation consiste à protéger les données.

- Syntaxe :

```
class Fille extends Mere { }
```

- Exemple :

```
class Compte {
        private double solde;
        public double getSolde() {
            return solde;
        }
        public void setSolde(double s) {
```

```
        solde = s;
    }
}
```

6. Héritage (Inheritance):

- Définition :

Permet à une classe de hériter d'une autre.

- Explication :

Permet de réutiliser le code.

- Exemple :

```
class Animal {
    void manger( ) {
        System.out.println("Mange");
    }
}
class Chien extends Animal {
    void aboyer() {
        System.out.println("Aboie");
    }
```

}

7. Polymorphisme:

• Définition :

Le polymorphisme permet d'utiliser une même méthode de différentes façons.

• Surcharge (Overloading) :

Même méthode, paramètres différents **.**

```
class Calcul {
 int addition(int a, int b) {
   return a + b;
 }
 int addition(int a, int b, int c) {
   return a + b + c;
 }
}
```

• Redéfinition (Overriding) :

Méthode redéfinie dans une classe fille .

```
class Animal {
 void son() {
   System.out.println("Son animal");
```

```
 }
}
class Chat extends Animal {
 void son() {
  System.out.println("Miaou");
 }
}
```

8. Abstraction:

• Définition :

L’abstraction permet de cacher les détails et montrer seulement l’essentiel.

• Classe abstraite :

```
abstract class Animal {
  abstract void son();
}
```

• Interface :

Une interface contient des methodes sans corps.

```
interface Animal {
 void son();
}
```

- Exemple :

```
interface Animal {
  void son();
}
class Chien implements Animal {
  public void son() {
    System.out.println("Aboie");
  }
}
```